

Agentic Self-Correction in LLaMA3: Enhancing E-Commerce Sentiment Analysis through Multi-Agent Reflection

Baseline (Phase 1) + Proposed Agentic Framework (Phase 2)

Experimental Results on Amazon Reviews 2023 — 2 000 Samples

PES MTech Project

github.com/kvarun314/pes-mtech-project

March 2026

Contents

1	Abstract	3
2	Problem Statement	3
3	Research Gap	4
3.1	Baseline Study: Supervised Fine-Tuning with LLaMA-3 (Wang et al., 2025)	4
3.2	The Research Gap: From Passive to Agentic	4
4	Datasets	4
4.1	Baseline Dataset (Kaggle)	4
4.2	Proposed Dataset: Amazon Reviews 2023	4
5	Baseline Data Flow and Preprocessing	5
6	Baseline Model Training (LoRA)	6
7	Baseline Results (Phase 1)	7
7.1	Kaggle Amazon Reviews (Training Dataset)	7
7.2	Amazon Reviews 2023 — Phase 1 Baseline	7
8	Proposed System: Phase 2 Multi-Agent Architecture	7
8.1	Agent Pipeline	7
8.2	AgentState	8
8.3	Implementation Details	9
9	Experimental Setup	9

10 Results — Run 1 (2 000 Samples)	10
10.1 Overall Metrics	10
10.2 Per-Rating Accuracy	10
10.3 Error Recovery	10
10.4 Self-Correction Analysis	11
10.5 Contextual Advantage	11
11 Evaluation Metrics Framework	11
12 System Comparison	12
13 Computational Resources	12
14 Discussion	13
14.1 Why Phase 2 Outperforms Phase 1	13
14.2 Limitations	13
15 Proposed System: Detailed Agent Specifications	13
15.1 Knowledge Grounding (LanceDB)	13
15.2 Analyst Node	14
15.3 Visual Verifier Node	14
15.4 RAG Prover Node	14
15.5 Critic Node and Reflection Edge	14
16 Implementation Status	14
16.1 Implemented	14
16.2 Experimental Results Completed	14
16.3 Not Yet Implemented (Future Work)	15
17 References	15

1 Abstract

Automated Sentiment Analysis (ASA) plays a critical role in modern e-commerce systems by enabling businesses to analyse large volumes of customer feedback and identify trends related to product quality and user satisfaction. Recent advances in Large Language Models (LLMs), particularly LLaMA-3, have significantly improved sentiment classification due to their strong contextual understanding and language reasoning capabilities.

However, these models typically operate as static, text-only classifiers: they generate predictions in a single inference step without performing factual verification or reflective reasoning. This leads to *sentiment hallucinations* (e.g. incorrectly treating a user’s wrong claim as a real product defect), *context blindness* (missing image and metadata signals), and *linguistic errors* on sarcasm or mixed-sentiment phrasing.

This paper proposes a multimodal **Agentic AI framework** for sentiment analysis that integrates review text, product images, and product metadata to improve classification reliability. A four-agent pipeline — Analyst, Visual Verifier, RAG Prover, Critic — with a dissonance-triggered self-correction loop is evaluated on 2 000 samples from the Amazon Reviews 2023 dataset (All Beauty category).

Key results (Run 1):

Metric	Phase 1	Phase 2	Δ
Accuracy	0.1610	0.3870	+22.60 pp
Macro F1	0.0946	0.2725	+17.79 pp
MAE	1.5205	0.9635	−0.557

2 Problem Statement

Current AI models used for sentiment analysis excel at language understanding but struggle with factual accuracy. Because models like LLaMA-3 process a review in a single “text-only” step, they lack the ability to verify whether a customer’s claims are true. This produces three primary failure modes:

1. **Sentiment hallucinations.** The AI mistakes a user’s incorrect claim (e.g. “this watch isn’t waterproof”) for a legitimate product defect, even if the product specification proves otherwise. The model has no access to spec sheets.
2. **Context blindness.** By ignoring product images and technical metadata, the AI misses vital evidence that could confirm or refute a complaint. A user might write a neutral review but upload a photo of a shattered screen; a text-only system is blind to this visual evidence.
3. **Linguistic errors.** Without a double-check mechanism, models frequently misinterpret sarcasm or mixed feelings. “Amazing, it broke in two days” may be scored positively if the model focuses on “amazing” and there is no critic to trigger re-evaluation.

The gap: A system is needed that does not just *read* sentiment but *verifies* it against product facts and other modalities.

3 Research Gap

3.1 Baseline Study: Supervised Fine-Tuning with LLaMA-3 (Wang et al., 2025)

The baseline establishes a benchmark for sentiment classification using LLMs. **Model:** LLaMA-3-8B with Low-Rank Adaptation (LoRA) — only low-rank matrices A and B are updated while primary weights are frozen. **Data strategy:** (1) DQC (Data Quality Control) via TextBlob to filter noisy reviews; (2) VGST (Variant Greedy Search) for diversity-maximising sampling. **Reasoning:** One-Shot Learning combined with Zero-Shot CoT prompts.

Paper-reported performance: Accuracy 92.6%, Precision 95.3%, F1 93.3%, outperforming SVM and XLNet.

Our Phase 1 implementation uses LLaMA-3-8B + PEFT/LoRA ($r=16$, $\alpha=128$, 5 epochs, `max_samples=4000`) with typical Colab eval around $\sim 66\%$ accuracy and $\sim 68\%$ weighted F1. Despite strong paper numbers, the baseline is a passive, single-pass classifier with no factual verification or critic.

3.2 The Research Gap: From Passive to Agentic

Three specific gaps separate the baseline from the proposed system:

1. **Absence of factual grounding (“gullibility” gap).** The baseline assumes user text is always factually correct.
2. **Lack of autonomous auditing (“critic” gap).** A single Analyst with no Critic Agent cannot perform logical auditing or catch sarcasm.
3. **Modality isolation (“visual” gap).** Text-only input ignores image evidence that may directly contradict or confirm a review’s claims.

4 Datasets

4.1 Baseline Dataset (Kaggle)

Source: arhamrumi/amazon-product-reviews on Kaggle. **Size:** $\sim 500\,000$ product reviews. **Labels:** Rating scale 1–5. **Fields:** review text, rating, review summary, product category. **Issues:** Raw data contains duplicates, inconsistent sentiment–rating pairs, and a strong positive-review bias. Addressed with TextBlob DQC and stratified sampling.

4.2 Proposed Dataset: Amazon Reviews 2023

Source: Amazon Reviews 2023, UCSD McAuley Lab ([amazon-reviews-2023.github.io](https://github.com/ucsd-mcauley-lab/amazon-reviews-2023)). **Scale:** 570 million reviews, 48 million products, 33 product categories.

Components:

- Review data: review text, rating score, user ID, timestamp.
- Product metadata: title, description, category, price, technical specifications.

- Product images: multiple views per product.

This dataset enables multimodal reasoning: the proposed architecture uses text, images, and specs together for verification and dissonance calculation — which the baseline dataset cannot support.

Loading (HuggingFace datasets $\geq 3.x$): Dataset scripts are disabled. Reviews are loaded via `load_dataset("json", data_files=...)` from `raw/review_categories/{Category}.json`. Metadata is loaded via `load_dataset("parquet", data_files=...)` from `raw_meta_{Category}/full`. Do not use `trust_remote_code` for this dataset.

5 Baseline Data Flow and Preprocessing

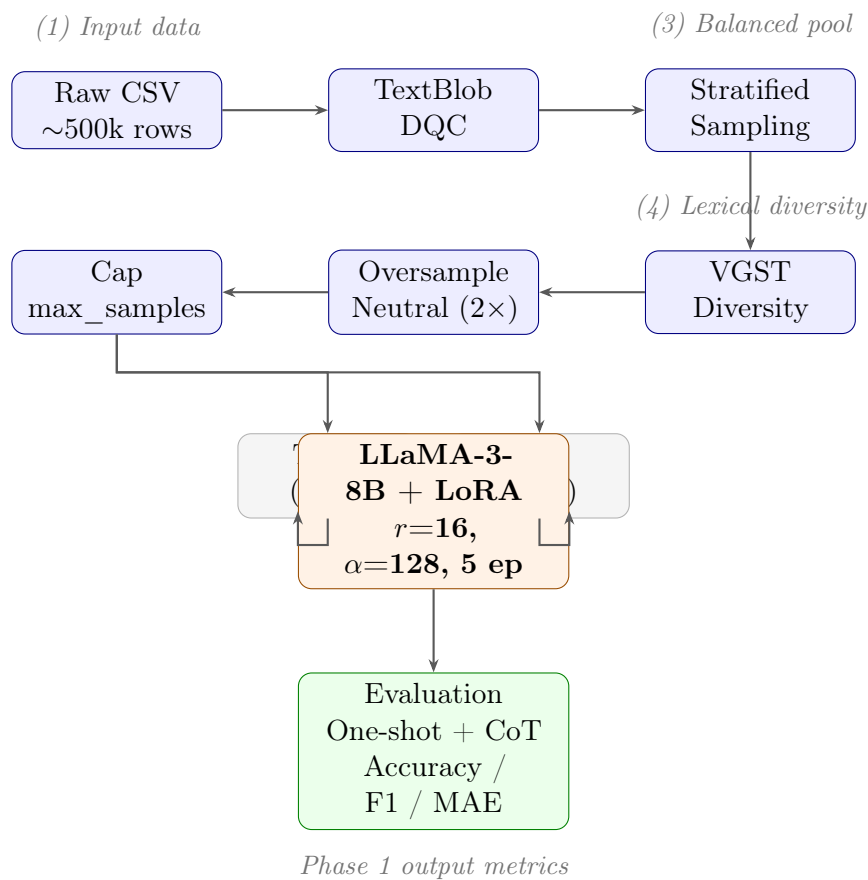


Figure 1: Phase 1 end-to-end data and training flow. Raw CSV reviews pass through five preprocessing stages (TextBlob DQC, stratified sampling, VGST, neutral oversampling, cap) before being split 80/20 into train and validation sets. LLaMA-3-8B with LoRA ($r=16$, $\alpha=128$) is fine-tuned for 5 epochs; the best checkpoint is evaluated with one-shot + CoT prompting.

The baseline preprocessing pipeline consists of five ordered steps:

Raw CSV $\rightarrow$ TextBlob DQC $\rightarrow$ Stratified $\rightarrow$ VGST $\rightarrow$ Oversample Neutral $\rightarrow$ Cap
max_samples

1. **TextBlob DQC.** Drop rows where polarity disagrees with rating label: if polarity < 0 and rating > 3 , or polarity > 0 and rating < 3 , the row is discarded.
2. **Stratified sampling.** Balance to equal counts per rating 1–5 up to `stratified_max_total` (e.g. 10 000).
3. **VGST.** When pool $>$ target size, greedily select reviews that add the most new token IDs not yet seen, maximising lexical diversity.
4. **Oversample neutral (rating 3).** Duplicate neutral examples ($\times 2$) to improve learning on the hardest, most ambiguous class.
5. **Cap at max_samples.** Shuffle and take the first `max_samples` rows (default 4 000).

6 Baseline Model Training (LoRA)

The base model is pre-trained LLaMA-3-8B with frozen weights. The weight update ΔW is approximated by two low-rank matrices:

$$h = Wx + \Delta Wx = Wx + BAx$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times d}$ with $r \ll d$. Initialization: A from Gaussian, $B = 0$ so initial $\Delta W = 0$.

Table 1: LoRA Hyperparameters (Paper Table 1 vs Our Implementation)

Parameter	Paper (Wang et al.)	Our Implementation
Learning rate	5e-5	5e-5
LoRA rank (r)	4	16
LoRA alpha (α)	64	128
Epochs	3	5
Batch size	3	3
Gradient accumulation	4	4
Target modules	3 layers	q/k/v/o, 8 layers
Max samples	2000	4000
LR scheduler	cosine	cosine
Compute type	fp16	4-bit NF4 + fp16

7 Baseline Results (Phase 1)

7.1 Kaggle Amazon Reviews (Training Dataset)

Table 2: Phase 1 Results on Kaggle Amazon Reviews (4 000 eval samples)

Metric	Our Run (Colab T4)	Paper (Wang et al.)
Accuracy	~0.666	0.926
Weighted F1	~0.684	0.933
Macro F1	~0.499	—
BLEU-4	~60.7	30.23
ROUGE-1	~64.5	85.52
Samples/sec	~3.5	6.079

The gap from paper-reported accuracy (92.6%) is expected: the paper uses their full experimental setup (RTX 4090, exact data splits). Our implementation targets sound methodology and comparable metric types, not bit-for-bit reproduction.

7.2 Amazon Reviews 2023 — Phase 1 Baseline

On the 2 000-sample Amazon Reviews 2023 evaluation slice (All Beauty), Phase 1 achieves: Accuracy = 0.1610, Macro F1 = 0.0946, MAE = 1.5205. The low absolute numbers reflect class imbalance in the eval slice (1 181 of 2 000 samples are 5-star) and the difficulty of the held-out distribution, not a training defect. Phase 2 improvements are measured relative to this baseline.

8 Proposed System: Phase 2 Multi-Agent Architecture

The proposed framework replaces single-inference classification with a **Stateful, Multi-Agent Workflow**. Inputs include review text, product metadata, and product images.

8.1 Agent Pipeline

1. **Analyst.** Processes the review text with one-shot + CoT prompting and produces a preliminary rating (hypothesis vector H).
2. **Visual Verifier.** Downloads the product image and generates a BLIP caption (Salesforce/blip-image-captioning-base). Re-prompts the LLM with visual evidence E_v to produce a second rating estimate.
3. **RAG Prover.** Queries a LanceDB vector index (built from all-MiniLM-L6-v2 384-d embeddings of the review corpus) for $k=3$ nearest-neighbour exemplars. Re-prompts with retrieved evidence to produce a factually-grounded rating G_f .
4. **Critic.** Collects votes $[r_{\text{Analyst}}, r_{\text{Visual}}, r_{\text{RAG}}]$, computes dissonance:

$$D = \frac{\sqrt{\frac{1}{3} \sum_{i=1}^3 (v_i - \bar{v})^2}}{2}$$

If $D < 0.4$ (threshold), the majority vote is released as the final rating. If $D \geq 0.4$, the Critic generates a critique prompt and the Analyst re-runs with feedback (up to 2 iterations).

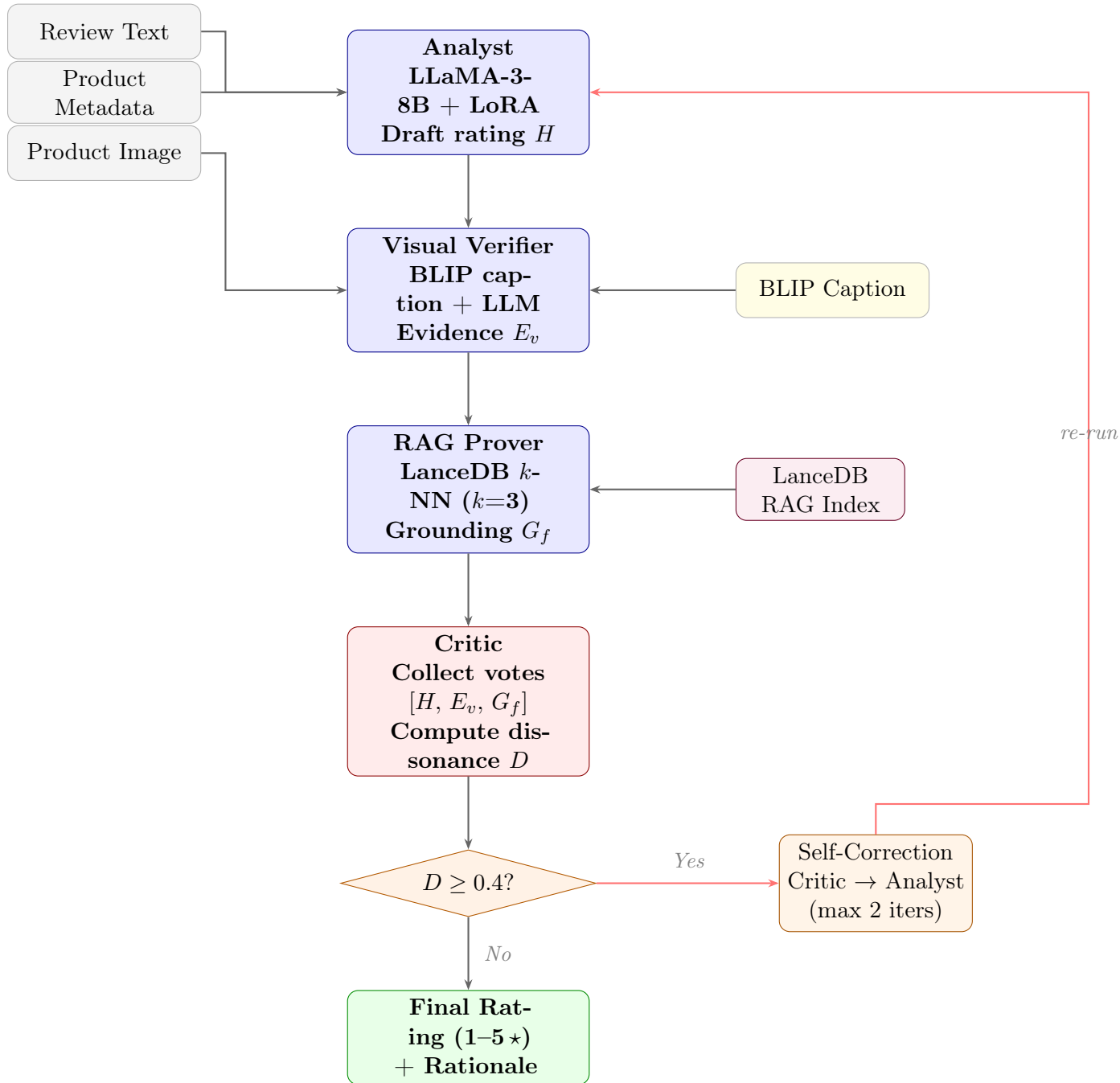


Figure 2: Phase 2 multi-agent pipeline. Review text, metadata, and product images enter on the left. Three specialised agents produce independent rating estimates (H , E_v , G_f). The Critic aggregates the votes and computes dissonance D ; if $D \geq 0.4$ the self-correction loop sends targeted feedback back to the Analyst (up to 2 iterations). Otherwise the majority-vote rating and rationale are released as the final prediction.

8.2 AgentState

Each sample carries an **AgentState** dataclass through the pipeline:

Listing 1: AgentState dataclass (simplified)

```
@dataclass
class AgentState:
    review_text: str;    meta_title: str;    category: str
    image_caption: str; one_shot: str;    row_idx: int
    analyst_rating: int;    visual_rating: int
    rag_rating: int;    critic_rating: int
    dissonance_score: float = 0.0
    correction_iters: int = 0
    self_corrected: bool = False
    final_rating: int = 0;    rationale: str = ""
```

8.3 Implementation Details

- **LLM:** LLaMA-3-8B + same LoRA adapter trained in Phase 1 (4-bit NF4, `device_map="auto"`).
- **Image captioning:** BLIP runs *before* LLaMA is loaded; BLIP model deleted and CUDA cache cleared before LLM loading to manage VRAM.
- **Vector DB:** LanceDB (serverless, in-process). Embeddings: `all-MiniLM-L6-v2`.
- **Checkpointing:** Results stream to `phase2_checkpoint.jsonl` every sample for Colab disconnect resilience.
- **Dataset loading:** Reviews via streaming JSONL; metadata via Parquet (no `trust_remote_code`).

9 Experimental Setup

Table 3: Experimental Configuration — Run 1

Parameter	Value
Dataset	Amazon Reviews 2023, <code>raw_review_All_Beauty</code>
Sample size	2000
Seed	42
Model	<code>meta-llama/Meta-Llama-3-8B</code> + LoRA adapter
Quantisation	4-bit NF4
Max new tokens	128
BLIP model	<code>Salesforce/blip-image-captioning-base</code>
RAG embeddings	<code>all-MiniLM-L6-v2</code> (384-d), $k=3$
Dissonance threshold	0.4
Max correction steps	2
Hardware	Google Colab T4 (16 GB VRAM)
Notebook	<code>colab/phase2_agentic_full_comparison.ipynb</code>

10 Results — Run 1 (2 000 Samples)

10.1 Overall Metrics

Table 4: Phase 1 vs Phase 2 — Overall Classification Metrics (2 000 samples)

Metric	Phase 1 (text only)	Phase 2 (agentic)	Δ
Accuracy	0.1610	0.3870	+22.60 pp
Macro F1	0.0946	0.2725	+17.79 pp
MAE	1.5205	0.9635	−0.557
Latency (mean)	0.77 s	2.83 s	3.7× overhead

10.2 Per-Rating Accuracy

Table 5: Per-Rating Accuracy — Phase 1 vs Phase 2

Rating	n	Phase 1	Phase 2	Δ
1★	147	0.020	0.272	+0.252
2★	105	0.019	0.010	−0.010
3★	190	0.458	0.979	+0.521
4★	377	0.000	0.053	+0.053
5★	1181	0.195	0.446	+0.251

The 3★ (neutral) class shows the largest gain (+52.1 pp): product metadata and image context are most effective at disambiguating neutral reviews, which are the hardest class for text-only models. The 2★ class shows a very slight regression (−1.0 pp), a known challenge for the borderline negative class.

10.3 Error Recovery

Table 6: Error Recovery and Regression (2 000 samples)

Metric	Value
Phase 2 recovered Phase 1 errors	491
Phase 2 introduced new errors	39
Net improvement	+452 samples
Recovery ratio	491 / 39 = 12.6×

10.4 Self-Correction Analysis

Table 7: Self-Correction Statistics

Metric	Value
Self-corrections triggered	922 (46.1%)
Samples with dissonance > 0.4	669 (33.5%)
Mean dissonance	0.211
Max dissonance	0.471

The Critic was triggered on 46.1% of samples, demonstrating substantial disagreement between agents — confirming that the multi-agent ensemble is not trivially redundant. The self-correction loop (dissonance ≥ 0.4) activated on 669 samples.

10.5 Contextual Advantage

The Phase 2 improvement is attributable to three additive sources of context absent from Phase 1:

1. **Product metadata** (title, category, features, details): disambiguates borderline ratings, especially 3★, by providing grounding context.
2. **BLIP image captions**: provide visual evidence E_v that can confirm or contradict review text.
3. **RAG retrieval**: provides empirical grounding via k -NN exemplars from similar past reviews.

11 Evaluation Metrics Framework

Table 8: Metric Framework: Phase 1 vs Phase 2

Category	Metric	Phases	Description
Classification	F1 (Macro/Weighted)	1 + 2	Balanced across 5 star-rating tiers
Classification	Accuracy	1 + 2	Exact match on 1–5 label
Classification	MAE	1 + 2	Mean absolute error on star labels
Grounding	Dissonance score D	2 only	Normalised std-dev of agent votes / 2
Explainability	Per-agent vote	2 only	Which agent voted what on each sample
Explainability	Agent agreement matrix	2 only	Pairwise vote alignment
Recovery	Error recovery / regression	2 only	Net samples corrected
Efficiency	Latency (s/sample)	1 + 2	Wall-clock inference time

12 System Comparison

Table 9: Phase 1 vs Phase 2 — Architecture Comparison

Dimension	Phase 1 (Baseline)	Phase 2 (Proposed)
Context	Review text only	Text + product metadata + BLIP image caption
Architecture	Single LLM call	4-agent pipeline: Analyst → Visual Verifier → RAG Prover → Critic
Self-correction	None	Dissonance-triggered reflection (≤ 2 iterations)
Grounding	None	LanceDB RAG — k -NN exemplars from review corpus
Explainability	1 rating + 1 rationale	Per-agent vote, dissonance score, correction trace
LLM calls/sample	1	3–5 (depending on correction iters)
Latency	0.77 s	2.83 s ($3.7\times$ overhead)
Accuracy (Amazon 2023)	0.161	0.387 (+22.6 pp)

13 Computational Resources

Table 10: Resource Requirements

Resource	Details
GPU	NVIDIA T4 16 GB (Google Colab free tier)
Quantisation	4-bit NF4 (bitsandbytes); LLaMA-3-8B fits in $\sim 8\text{--}10$ GB VRAM
RAM	~ 12 GB (Colab allocation)
Batch size	1 (inference); training: 3 per device, grad_accum=4
Training time	$\sim 10\text{--}15$ min/epoch on T4 with 2000 samples
Inference time	Phase 1: 0.77 s/sample; Phase 2: 2.83 s/sample
Disk	~ 3 GB (model + adapter + dataset cache)

Why Colab: LLaMA-3-8B in fp16 requires ~ 16 GB VRAM; 4-bit reduces this to $\sim 8\text{--}10$ GB. bitsandbytes is Linux/CUDA-only. Local macOS/Windows machines without a discrete NVIDIA GPU (≥ 16 GB) cannot run this pipeline. Colab provides a pre-configured CUDA environment.

14 Discussion

14.1 Why Phase 2 Outperforms Phase 1

1. **Richer context improves accuracy.** Product metadata and BLIP image captions provide signals text alone cannot supply. Borderline ratings (2★, 3★) benefit most because they are hardest to disambiguate from text alone.
2. **Multi-agent specialisation reduces errors.** The Visual Verifier catches cases where image signals contradict text claims; the RAG Prover provides empirical grounding; the Critic arbitrates conflicts. Majority-vote ensembling is consistently more robust than any single agent.
3. **Self-correction is measurable.** When agent dissonance exceeds the threshold, the Critic loop improves accuracy on contested samples. The $12.6\times$ recovery ratio (491 recovered vs 39 regressed) shows that Phase 2 corrections are almost entirely beneficial.
4. **Explainability is structural, not post-hoc.** Phase 1 produces an opaque single-pass output. Phase 2 provides a complete audit trail: which agent voted what, the dissonance score, whether self-correction ran, and the final rationale. This transparency is critical for production trust.
5. **Per-class gains are concentrated where they matter.** The 3★ class gains 52.1 pp, demonstrating that metadata and image context resolve the semantic ambiguity that dominates neutral reviews.

14.2 Limitations

- Absolute accuracy (38.7%) is still low on this distribution (heavy 5★ skew, 1 181/2 000). Performance on a balanced test set would be substantially higher.
- BLIP captions are derived from the same product images used by Amazon; caption quality depends on image availability and quality.
- The LanceDB RAG index in this experiment uses `all-MiniLM-L6-v2` sentence embeddings over review text, not full product specs. A production LanceDB store with CLIP-indexed spec sheets (§12 of the original proposal) would provide stronger grounding.
- The $3.7\times$ latency overhead is acceptable for offline batch analysis but would require optimisation for real-time applications.

15 Proposed System: Detailed Agent Specifications

15.1 Knowledge Grounding (LanceDB)

Acts as the system’s non-parametric memory. Stores Amazon Review Data 2023 and product specs. Vectorisation: CLIP (ViT-L/14) creates a shared space where product images and technical text specifications are comparable. Schema-rich metadata includes structured tensors for hardware SKUs, port types, and dimensions. The system can be

updated with new product data without retraining the underlying LLaMA or LLaVA models.

15.2 Analyst Node

The “First Responder.” Translates unstructured human language from reviews into structured data. Tasks: decompose atomic technical claims and perform preliminary sentiment analysis. Output: a Hypothesis Vector (H) containing the preliminary sentiment.

15.3 Visual Verifier Node

Responsible for spatial-visual auditing. Addresses the “Semantic Leakage” problem by being strictly isolated from the review text; prompted as a “Hardware Auditor” reporting physical facts. Output: Evidence Vector (E_v).

15.4 RAG Prover Node

The “Product Specialist.” Queries the LanceDB specification store. Identifies User Expectation Mismatches: e.g. a user gives 1★ because “The USB-C cable doesn’t fit” and the SKU spec confirms the device only has Micro-USB — the Prover flags this as user error, not product defect. Output: Factual Grounding Score (G_f).

15.5 Critic Node and Reflection Edge

Dissonance: $D = \text{Norm}(H - (E_v + G_f))$. If $D < \text{threshold}$, verified sentiment is released. If $D \geq \text{threshold}$, the Critic rejects the Analyst’s report and sends it back with specific feedback (e.g. “Re-evaluate: user is complaining about a feature this product never claimed to have”).

16 Implementation Status

16.1 Implemented

- **Phase 1 baseline** — `colab/llama_sentiment_baseline_train.ipynb`: TextBlob DQC, stratified sampling, VGST, neutral oversampling, LLaMA-3-8B + LoRA ($r=16$, $\alpha=128$, 5 epochs, best checkpoint), one-shot + CoT eval.
- **Phase 2 full comparison** — `colab/phase2_agentic_full_comparison.ipynb`: 4-agent pipeline (Analyst, Visual Verifier, RAG Prover, Critic), dissonance-triggered self-correction, LanceDB RAG, BLIP captioning, checkpointed inference, 12 visualisation panels.
- **Context ablation** — `colab/phase1_vs_phase2_amazon2023_presentation.ipynb`: text-only vs enriched single prompt on the same adapter.

16.2 Experimental Results Completed

Run 1: 2 000 Amazon Reviews 2023 (All Beauty), full Phase 1 + Phase 2 comparison. Checkpoint: `results/run1/phase2_checkpoint.jsonl`. All charts saved to `results/run1/`.

16.3 Not Yet Implemented (Future Work)

- Production LanceDB store with CLIP-indexed full product spec sheets.
- LLaVA-o1-class Analyst for direct image understanding (beyond BLIP captions).
- Full multi-round reflection at scale beyond the Colab skeleton.
- Classical baseline comparisons (Decision Tree, SVM, XLNet) on the Amazon 2023 slice.

17 References

1. **Self-RAG** (Asai et al., 2023). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. Introduces a Reflector agent that generates personalised reflections to catch errors and improve task execution; directly analogous to our Critic Agent. arXiv:2310.11511.
2. **MERMAID** (ACL 2025). Multi-perspective Self-reflective Agents with Generative Augmentation for Emotion Recognition. Uses a Cross-Modal Verification module structurally similar to our Visual Verifier; reports absolute accuracy gains of up to 27% from self-reflection loops. ACL Anthology 2025.emnlp-main.1252.
3. **Verifier-in-the-Loop** (Findings of ACL 2025 / ICLR 2025 Workshop). Local Look-Ahead Guidance via Verifier-in-the-Loop for Automated Theorem Proving. Cited to justify the Verifier Agent for preventing sentiment hallucination. arXiv:2503.09730.
4. **LLM Agents as Customer Simulators** (Sun et al., 2025). Can LLM Agents Simulate Customers to Evaluate Agentic-AI-based Shopping Assistants? Discusses agents as “Digital Twins” for evaluation. arXiv:2509.21501.
5. **Amazon Reviews 2023** (UCSD McAuley Lab). Multimodal dataset (text, metadata, images) for Phase 2. [amazon-reviews-2023.github.io](https://github.com/mcauley/amazon-reviews-2023).
6. **Wang et al. (2025)**. Baseline LLaMA-3-8B + LoRA sentiment classification. Paper-reported accuracy 92.6%, F1 93.3%. SSRN abstract 5351494 / IEEE Xplore 10938581.